

CPT204_期末复习

哈希

哈希用于sets和map

要最终得到index, 我们首先需要使用 hash 函数将搜索键Search key转换为整数值哈希码hash code, 然后将哈希码压缩成哈希表hash table的索引。

- Hash function : Search Key $\rightarrow$ Hash Code (as integer)
- Compressing: Hash Code (as integer) $\rightarrow$ Index

压缩哈希码 Compressing Hash Codes

键的哈希代码可以是超出哈希表索引范围的大整数,例如, 一个大小为 11 的表 (索引 0-10), 哈希码可能是一个大整数, 如 366712642.我们需要缩小它以适应指数的范围

假设哈希表的索引介于 0 和 N-1 之间 -- 缩放整数的最常见方法是:

$h(\text{hashCode}) = \text{hashCode} \% N$

其中%是取模,N一般取一个素数以确保均匀分布.

哈希碰撞 Handling Collisions

当两个不同的key映射到哈希表中的同一个index时,发生哈希冲突

开放寻址(Open addressing)是发生冲突后寻找open location 的方法

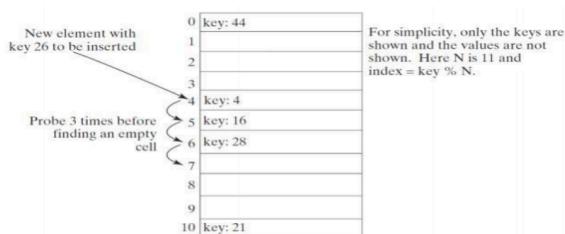
常用的有:linear probing, quadratic probing, double hashing

Separate chaining将具有相同哈希索引的所有条目放置在列表中的相同位置

Linear Probing

在将条目插入到哈希表期间发生冲突时, 线性探测会按顺序查找下一个可用位置

如果 $\text{hashTable}[\text{key} \% N]$ 发生冲突, 检查 $\text{hashTable}[(\text{key}+1) \% N]$ 是否可用,如果没有, 检查 $\text{hashTable}[(\text{key}+2) \% N]$, 直到有可用的单元格.



要从哈希表中删除条目，搜索与 key 匹配的条目,如果找到该条目，则放置一个特殊标记，以表示该条目已被删除，但可用于插入其他值

哈希表中的每个单元格都有三种可能的状态：occupied、marked 或 empty

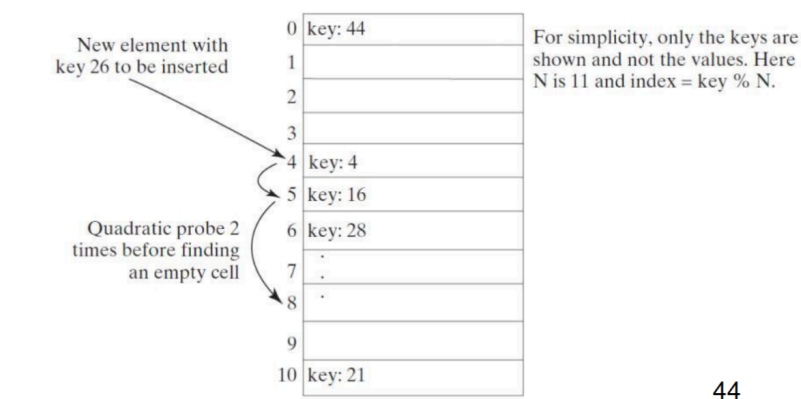
线性探测往往会导致哈希表中的连续单元组被占用 —— 每组称为一个簇 cluster

因为当发生碰撞时，新元素总是被放置在附近的空槽中 ($k \% N$, $(k+1) \% N$, $(k+2) \% N$...)，导致被占用的单元积累并形成聚类

每个集群实际上都是一个探测序列，您在检索、添加或删除条目时必须搜索该序列。这是线性探测的一大缺点

Quadratic Probing

二次探测查看索引 $(key + j^2) \% N$ 处的单元格,对 $j \geq 0$,



如图,每次跳更大.

二次探测的工作方式与用于检索、插入和删除的线性探测相同，但搜索序列发生了变化（查找打开位置所需的“步骤”）

它仍然有自己的集群问题，称为二级簇secondary clustering，用于使用相同的探测序列与占用的条目碰撞

[0]	44	
[1]		
[2]		$(15+3^2)\%11=2$
[3]		
[4]	4	$15\%11=4$
[5]	16	$(15+1^2)\%11=5$
[6]	28	
[7]		
[8]	26	$(15+2^2)\%11=8$
[9]		
[10]	21	But what if add 37

Double Hashing

双重哈希对键使用辅助哈希函数 $h'(key)$ 来确定增加值,从而避免碰撞

双重哈希查看索引 $(h(key) + j * h'(key)) \% N$ ($j \geq 0$) 处的单元格, 即

$(h(key) + 0 * h'(key)) \% N$, $(h(key) + 1 * h'(key)) \% N$, $(h(key) + 2 * h'(key)) \% N$,
 $(h(key) + 3 * h'(key)) \% N$, and so on

如: $h(key) = key \% 11$;

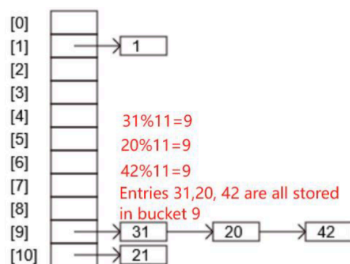
$h'(key) = 7 - key \% 7$;

$h'(key)$ 可以是其他公式, 只要其结果确保步长永远不会为零, 并且相对于表大小 (N) 互质, 例如, $6 - key \% 6$, 步长可以是 1,2,3,4,5,6, 而 1,2,3,4,5,6 相对质数为 11

Separate Chaining

将具有相同哈希索引的所有条目放置在同一位置, 而不是查找新位置. 单独链接方案中的每个位置都称为存储桶bucket, 可以使用数组、ArrayList 或 LinkedList (在下面的示例中) 来实现

存储桶是包含多个条目的容器:



负载因子和重新哈希 Load Factor and Rehashing

负载因子 λ (lambda) 是元素数与哈希表大小的比率, $\lambda = n / N$ 其中 n 表示元素数, N 表示哈希表中的位置数

对于开放寻址方案, λ 介于 0 和 1 之间, 空=0, 满=1

对于单独的链接方案, λ 可以是任何值

随着 λ 的增加, 碰撞的概率增加因为负载因子衡量的是哈希表的填充程度

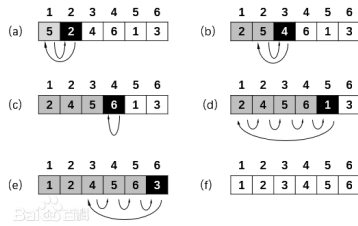
对于开放寻址方案, 应将负载因子保持在 0.5 以下, 对于单独的链接方案, 应将负载因子保持在 0.9 以下

java.util.HashMap的默认阈值是0.75

值传递和引用传递 Pass By Value/ Reference

■ 排序:

插入排序:每次从未排序部分中取第一个插入到已排序部分中的正确位置



选择排序:每次选择最小或最大的放到已排序区

堆排序:

快速排序:

■ Java实现六大经典排序算法

以下是Java实现的冒泡排序、选择排序、插入排序、堆排序、快速排序和归并排序的完整代码。

■ 1. 冒泡排序 (Bubble Sort)

```
1 public class BubbleSort {
2     public static void bubbleSort(int[] arr) {
3         int n = arr.length;
4         for (int i = 0; i < n - 1; i++) {
5             boolean swapped = false;
6             for (int j = 0; j < n - i - 1; j++) {
7                 if (arr[j] > arr[j + 1]) {
8                     // 交换相邻元素
9                     int temp = arr[j];
10                    arr[j] = arr[j + 1];
11                    arr[j + 1] = temp;
12                    swapped = true;
13                }
14            }
15            // 如果没有发生交换, 说明数组已经有序
16            if (!swapped) break;
17        }
18    }
19
20    public static void main(String[] args) {
21        int[] arr = {64, 34, 25, 12, 22, 11, 90};
22        bubbleSort(arr);
23        System.out.println("冒泡排序结果: " + Arrays.toString(arr));
24    }
25 }
```

```
24     }
25 }
```

■ 2. 选择排序 (Selection Sort)

```
1  public class SelectionSort {
2      public static void selectionSort(int[] arr) {
3          int n = arr.length;
4          for (int i = 0; i < n - 1; i++) {
5              int minIdx = i;
6              for (int j = i + 1; j < n; j++) {
7                  if (arr[j] < arr[minIdx]) {
8                      minIdx = j;
9                  }
10             }
11             // 交换找到的最小元素
12             int temp = arr[minIdx];
13             arr[minIdx] = arr[i];
14             arr[i] = temp;
15         }
16     }
17
18     public static void main(String[] args) {
19         int[] arr = {64, 25, 12, 22, 11};
20         selectionSort(arr);
21         System.out.println("选择排序结果: " + Arrays.toString(arr));
22     }
23 }
```

■ 3. 插入排序 (Insertion Sort)

```
1  public class InsertionSort {
2      public static void insertionSort(int[] arr) {
3          int n = arr.length;
4          for (int i = 1; i < n; i++) {
5              int key = arr[i];
6              int j = i - 1;
7
8              // 将大于key的元素向后移动
9              while (j >= 0 && arr[j] > key) {
10                 arr[j + 1] = arr[j];
11                 j--;
12             }
13             arr[j + 1] = key;
14         }
15     }
16 }
```

```

14     }
15 }
16
17 public static void main(String[] args) {
18     int[] arr = {12, 11, 13, 5, 6};
19     insertionSort(arr);
20     System.out.println("插入排序结果: " + Arrays.toString(arr));
21 }
22 }

```

■ 4. 堆排序 (Heap Sort)

```

1  public class HeapSort {
2      public static void heapSort(int[] arr) {
3          int n = arr.length;
4
5          // 构建最大堆
6          for (int i = n / 2 - 1; i >= 0; i--) {
7              heapify(arr, n, i);
8          }
9
10         // 一个个从堆顶取出元素
11         for (int i = n - 1; i > 0; i--) {
12             // 将当前根节点(最大值)与末尾元素交换
13             int temp = arr[0];
14             arr[0] = arr[i];
15             arr[i] = temp;
16
17             // 重新调整堆
18             heapify(arr, i, 0);
19         }
20     }
21
22     private static void heapify(int[] arr, int n, int i) {
23         int largest = i; // 初始化最大值为根节点
24         int left = 2 * i + 1;
25         int right = 2 * i + 2;
26
27         // 如果左子节点大于根节点
28         if (left < n && arr[left] > arr[largest]) {
29             largest = left;
30         }
31
32         // 如果右子节点大于当前最大值
33         if (right < n && arr[right] > arr[largest]) {
34             largest = right;

```

```

35     }
36
37     // 如果最大值不是根节点
38     if (largest != i) {
39         int swap = arr[i];
40         arr[i] = arr[largest];
41         arr[largest] = swap;
42
43         // 递归调整受影响的子堆
44         heapify(arr, n, largest);
45     }
46 }
47
48 public static void main(String[] args) {
49     int[] arr = {12, 11, 13, 5, 6, 7};
50     heapSort(arr);
51     System.out.println("堆排序结果: " + Arrays.toString(arr));
52 }
53 }

```

■ 5. 快速排序 (Quick Sort)

```

1  public class QuickSort {
2      public static void quickSort(int[] arr) {
3          quickSort(arr, 0, arr.length - 1);
4      }
5
6      private static void quickSort(int[] arr, int low, int high) {
7          if (low < high) {
8              // 找到分区点
9              int pi = partition(arr, low, high);
10
11              // 递归排序分区前后的子数组
12              quickSort(arr, low, pi - 1);
13              quickSort(arr, pi + 1, high);
14          }
15      }
16
17      private static int partition(int[] arr, int low, int high) {
18          int pivot = arr[high]; // 选择最后一个元素作为基准
19          int i = low - 1; // 小于基准的元素的索引
20
21          for (int j = low; j < high; j++) {
22              // 如果当前元素小于基准
23              if (arr[j] < pivot) {
24                  i++;

```

```

25         // 交换arr[i]和arr[j]
26         int temp = arr[i];
27         arr[i] = arr[j];
28         arr[j] = temp;
29     }
30 }
31
32 // 交换arr[i+1]和基准
33 int temp = arr[i + 1];
34 arr[i + 1] = arr[high];
35 arr[high] = temp;
36
37 return i + 1;
38 }
39
40 public static void main(String[] args) {
41     int[] arr = {10, 7, 8, 9, 1, 5};
42     quickSort(arr);
43     System.out.println("快速排序结果: " + Arrays.toString(arr));
44 }
45 }

```

■ 6. 归并排序 (Merge Sort)

```

1  public class MergeSort {
2      public static void mergeSort(int[] arr) {
3          mergeSort(arr, 0, arr.length - 1);
4      }
5
6      private static void mergeSort(int[] arr, int left, int right) {
7          if (left < right) {
8              // 找到中间点
9              int mid = left + (right - left) / 2;
10
11              // 分别对左右两部分排序
12              mergeSort(arr, left, mid);
13              mergeSort(arr, mid + 1, right);
14
15              // 合并已排序的两部分
16              merge(arr, left, mid, right);
17          }
18      }
19
20      private static void merge(int[] arr, int left, int mid, int right) {
21          // 计算两个子数组的大小
22          int n1 = mid - left + 1;

```



```

23         int n2 = right - mid;
24
25         // 创建临时数组
26         int[] L = new int[n1];
27         int[] R = new int[n2];
28
29         // 拷贝数据到临时数组
30         System.arraycopy(arr, left, L, 0, n1);
31         System.arraycopy(arr, mid + 1, R, 0, n2);
32
33         // 合并临时数组
34         int i = 0, j = 0;
35         int k = left;
36         while (i < n1 && j < n2) {
37             if (L[i] <= R[j]) {
38                 arr[k] = L[i];
39                 i++;
40             } else {
41                 arr[k] = R[j];
42                 j++;
43             }
44             k++;
45         }
46
47         // 拷贝L剩余元素
48         while (i < n1) {
49             arr[k] = L[i];
50             i++;
51             k++;
52         }
53
54         // 拷贝R剩余元素
55         while (j < n2) {
56             arr[k] = R[j];
57             j++;
58             k++;
59         }
60     }
61
62     public static void main(String[] args) {
63         int[] arr = {12, 11, 13, 5, 6, 7};
64         mergeSort(arr);
65         System.out.println("归并排序结果: " + Arrays.toString(arr));
66     }
67 }

```

■ 算法复杂度比较

排序算法	平均时间复杂度	最坏时间复杂度	空间复杂度	稳定性
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定

这些排序算法各有优缺点，适用于不同的场景。例如，对于小规模数据，插入排序可能比快速排序更高效；而对于大规模数据，归并排序和堆排序通常表现更好。

■ 类

数据字段data field, 构造器constructor, 方法method

```
class Circle {
    /** The radius of this circle */
    private double radius = 1.0;

    /** Construct a circle object */
    public Circle() {
    }

    /** Construct a circle object */
    public Circle(double newRadius) {
        radius = newRadius;
    }

    /** Return the area of this circle */
    public double getArea()
        return radius * radius * 3.14159;
    }
}
```

Diagram illustrating the components of the `Circle` class:

- Data field:** `private double radius = 1.0;`
- Constructors:** `public Circle() { }` and `public Circle(double newRadius) { radius = newRadius; }`
- Method:** `public double getArea() { return radius * radius * 3.14159; }`

■ 方法

在 Java 中，方法或构造器的**第一行**（即方法体前的声明部分）可以包含多种组合，具体取决于方法的类型（如实例方法、静态方法、构造器）和功能需求（如泛型、注解、访问控制等）。以下是所有可能的搭配及其语法格式：

1. 方法的声明格式

通用结构

```
1  [注解] [访问修饰符] [static] [final|abstract|synchronized] [<泛型参数>] 返回类型  
   方法名([参数列表]) [throws 异常列表] {  
2      // 方法体  
3  }
```

合法组合示例

(1) 普通实例方法

```
1  public int calculate(int a, int b) { ... }
```

(2) 静态方法

```
1  public static void print(String message) { ... }
```

(3) 泛型方法

```
1  public <T> T getFirst(List<T> list) { ... }
```

(4) 抛出异常的方法

```
1  public void readFile(String path) throws IOException { ... }
```

(5) 带注解的方法

```
1  @Override  
2  public String toString() { ... }
```

(6) final/synchronized 方法

```
1 public final synchronized void updateCounter() { ... }
```

(7) 抽象方法（无方法体）**

```
1 public abstract void draw();
```

(8) 可变参数方法**

```
1 public static void log(String... messages) { ... }
```

2. 构造器的声明格式**

通用结构**

```
1  [注解] [访问修饰符] 类名([参数列表]) [throws 异常列表] {  
2      // 构造器体  
3  }
```

合法组合示例

(1) 默认构造器**

```
1 public Person() { ... }
```

(2) 带参数的构造器**

```
1 public Person(String name, int age) { ... }
```

(3) 私有构造器（单例模式）**

```
1 private Person() { ... }
```

(4) 抛出异常的构造器**

```
1 public Person(String configFile) throws IOException { ... }
```

(5) 泛型构造器（在泛型类中）**

```
1 public class Box<T> {  
2     public Box(T value) { ... }  
3 }
```

(6) 带注解的构造器**

```
1 @Inject  
2 public Person(Service service) { ... }
```

|| **3. 特殊情况的搭配**

|| **(1) 方法/构造器 + 泛型 + 注解 + 异常**

```
1 @Transactional  
2 public <T extends Comparable<T>> T max(List<T> list) throws  
   IllegalArgumentException { ... }
```

|| **(2) 构造器 + 访问控制 + 异常**

```
1 protected DatabaseConnection(String url) throws SQLException { ... }
```

|| **(3) `static` + `final` + 泛型方法**

```
1 public static final <K, V> Map<K, V> emptyMap() { ... }
```

|| **(4) `synchronized` + 注解**

```
1 @GuardedBy("lock")  
2 public synchronized void criticalMethod() { ... }
```

|| **4. 不允许的搭配**

- ✗ 构造器不能有返回类型（包括 `void`）。
 - ✗ 构造器不能是 `abstract`、`final`、`static`（但可以是 `private`）。
 - ✗ 方法不能同时用 `final` 和 `abstract`（矛盾）。
 - ✗ 接口方法不能是 `final`（默认 `public abstract`）。
-

|| 总结

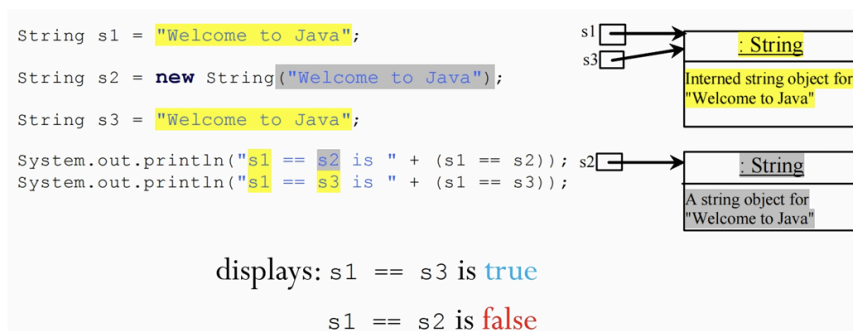
组件	方法支持	构造器支持
访问修饰符	✓	✓
<code>static</code>	✓	✗
<code>final</code>	✓	✗
<code>abstract</code>	✓	✗
<code>synchronized</code>	✓	✗
泛型 <code><T></code>	✓	✗（仅泛型类可用）
注解	✓	✓
<code>throws</code>	✓	✓
可变参数 <code>...</code>	✓	✓

Java 的方法和构造器声明非常灵活，但必须符合语法规则。你可以根据需求组合这些元素，但要注意某些限制（如构造器不能是 `static` 或 `final`）。

|| 字符串

intern 方法

String interning is a method of storing only one copy of each distinct compile-time constant/explicit string in the source code stored in a string intern pool (e.g., s1 and s3). 字符串暂存 是一种在存储在字符串暂存池（例如 s1 和 s3）中的源代码中仅存储每个不同编译时常量/显式字符串的一个副本的方法。 If we use the new operator, then a new object is created in heap (e.g., s2). 如果我们使用 new 运算符，则会在堆中创建一个新对象无论此时字符串常量池中是否存在与之相同的字符（例如，s2）。



String的 equals 方法已经被重写为只对比两个对象的值，而不是地址

比较器

自定义数据类实现比较器

下面我将演示如何创建一个自定义数据类，并实现比较器（Comparator）和可比较（Comparable）接口，使对象能够进行排序和比较。

1. 基本数据类实现

首先创建一个表示学生的数据类：

```
1 import java.util.Objects;
2
3 public class Student implements Comparable<Student> {
4     private final int id;
5     private final String name;
6     private final double gpa;
7     private final int age;
8
9     public Student(int id, String name, double gpa, int age) {
10         this.id = id;
11         this.name = name;
12         this.gpa = gpa;
13         this.age = age;
14     }
15
16     // Getters
17     public int getId() { return id; }
18     public String getName() { return name; }
19     public double getGpa() { return gpa; }
20     public int getAge() { return age; }
21
22     // 重写equals和hashCode
23     @Override
```

```

24     public boolean equals(Object o) {
25         if (this == o) return true;
26         if (o == null || getClass() != o.getClass()) return false;
27         Student student = (Student) o;
28         return id == student.id &&
29             Double.compare(student.gpa, gpa) == 0 &&
30             age == student.age &&
31             Objects.equals(name, student.name);
32     }
33
34     @Override
35     public int hashCode() {
36         return Objects.hash(id, name, gpa, age);
37     }
38
39     // 实现Comparable接口, 默认按ID排序
40     @Override
41     public int compareTo(Student other) {
42         return Integer.compare(this.id, other.id);
43     }
44
45     @Override
46     public String toString() {
47         return "Student{" +
48             "id=" + id +
49             ", name='" + name + '\'' +
50             ", gpa=" + gpa +
51             ", age=" + age +
52             '}';
53     }
54 }

```

■ 2. 实现多个比较器

创建不同的比较器实现不同的排序方式:

```

1  import java.util.Comparator;
2
3  public class StudentComparators {
4      // 按姓名排序
5      public static Comparator<Student> byName() {
6          return Comparator.comparing(Student::getName);
7      }
8
9      // 按GPA降序排序
10     public static Comparator<Student> byGpaDesc() {

```



```

11         return (s1, s2) -> Double.compare(s2.getGpa(), s1.getGpa());
12     }
13
14     // 按年龄排序, 年龄相同再按姓名排序
15     public static Comparator<Student> byAgeThenName() {
16         return Comparator.comparingInt(Student::getAge)
17             .thenComparing(Student::getName);
18     }
19 }

```

3. 使用示例

```

1  import java.util.ArrayList;
2  import java.util.Collections;
3  import java.util.List;
4
5  public class Main {
6      public static void main(String[] args) {
7          List<Student> students = new ArrayList<>();
8          students.add(new Student(3, "张三", 3.8, 20));
9          students.add(new Student(1, "李四", 3.5, 22));
10         students.add(new Student(2, "王五", 4.0, 19));
11         students.add(new Student(4, "赵六", 3.9, 20));
12
13         // 默认排序 (按ID, 实现了Comparable接口)
14         Collections.sort(students);
15         System.out.println("按ID排序:");
16         students.forEach(System.out::println);
17
18         // 使用比较器按姓名排序
19         students.sort(StudentComparators.byName());
20         System.out.println("\n按姓名排序:");
21         students.forEach(System.out::println);
22     }
23 }

```

关键点总结

1. ****实现Comparable接口****: 定义对象的自然排序顺序
2. ****创建Comparator****: 实现不同的排序逻辑
3. ****重写equals和hashCode****: 确保对象比较和哈希计算的正确性
4. ****使用Java 8+的Comparator方法****: 如 `comparing()`、`thenComparing()` 等简化代码

5. ****考虑使用记录类**** (Java 16+) : 简化数据类的定义

这种实现方式既保持了代码的灵活性（多种排序方式），又确保了类型安全和良好的面向对象设计。

■ 自动装箱 Automatic Unboxing

```
1 public class AutomaticUnboxing {
2     public static void main (String[] args) {
3         Integer num = null;
4         int x = num;
5         System.out.println(x);
6     }
7 }
```

这里第四行会产生运行时错误

■ 两种泛型约束写法的区别

■ 写法一：<E extends Comparable<E>>

```
1 public static <E extends Comparable<E>> E max(E a, E b)
```

- 表示类型 `E` 必须能够与**同类型**的其他实例比较
- 这是类型安全的，确保只能比较相同类型的对象

■ 写法二：<E extends Comparable>

```
1 public static <E extends Comparable> E max(E a, E b)
```

- 表示类型 `E` 实现了原始类型 `Comparable` 接口
- 会失去泛型类型检查，允许与任意对象比较
- 容易引发 `ClassCastException`

■ 八皇后问题

```
1 public class EightQueens {
2     private static final int N = 8; // 棋盘大小
3     private static int[] queens = new int[N]; // 皇后位置, queens[i]表示第i行皇后所在的列
4
5     public static void main(String[] args) {
6         solve(0); // 从第0行开始放置皇后
7     }
8
9     /**
10     * 递归放置皇后
11     * @param row 当前要放置皇后的行
12     */
13     private static void solve(int row) {
14         if (row == N) { // 所有行都已放置皇后
15             printSolution();
16             return;
17         }
18
19         for (int col = 0; col < N; col++) {
20             if (isSafe(row, col)) { // 检查当前位置是否安全
21                 queens[row] = col; // 放置皇后
22                 solve(row + 1); // 递归放置下一行的皇后
23             }
24         }
25     }
26
27     /**
28     * 检查在(row, col)位置放置皇后是否安全
29     */
30     private static boolean isSafe(int row, int col) {
31         for (int i = 0; i < row; i++) {
32             // 检查列冲突和对角线冲突
33             if (queens[i] == col ||
34                 queens[i] - i == col - row ||
35                 queens[i] + i == col + row) {
36                 return false;
37             }
38         }
39         return true;
40     }
41
42     /**
43     * 打印解决方案
```

```

44     */
45     private static void printSolution() {
46         System.out.println("解决方案 " + (solutionCount++) + ":");
47         for (int i = 0; i < N; i++) {
48             for (int j = 0; j < N; j++) {
49                 System.out.print(queens[i] == j ? "Q " : ". ");
50             }
51             System.out.println();
52         }
53         System.out.println();
54     }
55
56     private static int solutionCount = 1;
57 }

```

AVL的旋转

失衡类型	条件	旋转方式
LL	左子树高度 > 右子树 + 1, 且左子树的左子树高度 ≥ 右子树高度	左单旋
RR	右子树高度 > 左子树 + 1, 且右子树的右子树高度 ≥ 左子树高度	右单旋
LR	左子树高度 > 右子树 + 1, 且左子树的右子树高度 > 左子树高度	先右后左双旋
RL	右子树高度 > 左子树 + 1, 且右子树的左子树高度 > 右子树高度	先左后右双旋

